

编译原理 Project 2 实验报告

231275036 朱晗

April 24, 2026

1 实验概况

我的 Task Number 为 17, 需要实现 3.3: 把名等价改为结构等价重新做 1-17 的检查

2 编译运行

我使用了课程网站提供的 Makefile, 在 Code 目录下输入 `make` 即可编译形成 `Code/parser` 可执行文件.

3 编程实现

本次实验, 我新建了 `semantics.c` 和 `semantics.h` 来支持语义分析. 具体是以 `syntax.y` 导出的 AST 作为输入, 在其上遍历输出.

4 语义分析

4.1 符号表的实现

当前的符号表是以 Hash 表实现的, 符号表即一个 `Symbol` 类型的数组. 参考实验手册的方案, 使用了 `hashpjw` 作为 hash 函数

4.2 数据结构

- Symbol 的组织
 - name: 字符串类型, 存储符号名字

- info: 一个联合体类型, 存储 Type 或 FunctionInfo, 取决于 Symbol 的类型
- next: Symbol* 类型, For Hash Table Chaining 的链表指针
- depth: 为了作用域引入.(在我的要求中暂时不要求实现)
- FunctionInfo 的组织
 - returnType: Type 类型, 存储返回值
 - paramCount: int 类型, 存储参数数量
 - params: FieldList 类型, 存储参数列表
 - defined: bool 类型, 指示函数是否被定义过
- FieldList 的组织: 记录 Field 的名字和类型和下一个 FieldList
- Type 的组织
 - TypeKind kind: 指示 Type 的类型
 - union u: 包含不同类型 (BASIC, ARRAY, STRUCTURE) 的信息

在以上数据结构确定后, 只需要对不同语义处理 (定义, 声明, 表达式) 进行处理就可以完成任务. 具体来说, 程序引入了 bool 变量 `inFirstPass` 来指示扫描阶段. 如果是第一次扫描, 则进行插表等维护操作, 这时候可以发现重复定义等语义错误. 在符号表维护完毕后, 程序进行第二次扫描, 会检查赋值语句、运算语句、结构体成员访问语句等的语义合法性. 总体两次扫描解决问题

4.3 分析过程-函数调用栈

在 main 函数中, 我接入了 `analyzeSemantics(TreeNode* root)` 函数, 其中 `root` 参数传入的是 AST 的根. 这个函数主要完成了:

- `initSemantics`: init the symbol table, alloc memory for hash table.
- set `inFirstPass` to true, do `traverseExtDefList`.
- set `inFirstPass` to false, do `traverseExtDefList`.

`ExtDefList` 是最外层的 AST 结点, 我们会递归的逐层分析整个语法树.

5 选做内容：结构等价

所谓结构等价 (Structure Equivalent), 就是在两个结构体的类型检查中, 不去检查成员变量的名字, 而是只检查类型, 来做到结构上的等价. 名等价 (Name Equivalent) 要更加严格一些.

修改 `fieldListsEqual` 即可 (因为结构体等价, 等价于考虑其 `fieldLists` 的等价).

6 END

实践内容二到此结束